The order-book-imbalance signal (market data to trade)

Learning notes for `src/signal_bench.cpp`. The matching engine gave us a book you can *trade against*. A **signal** closes the loop: read the book, decide, and `submit` an order - then measure how fast that whole path runs. This one uses **order-book imbalance (OBI)**, one of the oldest and most robust microstructure signals.

---

1. The intuition

A limit order book quotes two queues: buyers resting on the **bid**, sellers resting on the **ask**. When the resting size is lopsided, the thin side tends to get eaten first, so the mid-price drifts *toward the thin side*:

- **More size on the bid than the ask** → buyers are eager, sellers are scarce → price tends to tick **up**. A short-horizon trader wants to **buy** now.
- **More size on the ask** → the mirror: price tends to tick **down** → **sell**.

It's a pressure gauge, not a crystal ball - a well-known effect that decays in milliseconds, which is exactly why the *latency* of acting on it matters.

---

2. The number

Sum the resting quantity over the best **K** price levels on each side (here K = 5), then:

$$OBI = \frac{bidDepth - askDepth}{bidDepth + askDepth} \quad \text{in} \quad [-1, +1]$$

`OBI = +1` means all the size is on the bid; `-1` means all on the ask; `0` is balanced. We only act on a **conviction** move, so we gate on a threshold `T = 0.30`:

- `OBI > +0.30` → aggressive **BUY** (take the offer)
- `OBI < -0.30` → aggressive **SELL** (hit the bid)
- otherwise → do nothing

Top-5 depth each side -> imbalance -> decision

OBI = (820 − 300) / (820 + 300) = 520 / 1120 = **+0.46**

+0.46 > +0.30 threshold -> **BUY** (take the offer)

Bids outweigh asks ~2.7:1 over the top 5 levels — lean long before the thin ask lifts.

---

3. Reading the depth cheaply

The whole point of the array-of-levels book is that this read is fast. `top_depth` walks out from the maintained touch (`best_bid_` / `best_ask_`), summing the first K *occupied* levels - O(K), no tree, no allocation:

```
uint64_t top_depth(Side side, int levels) const {
    uint64_t sum = 0; int found = 0;
    if (side == Side::Buy)
        for (int32_t t = best_bid_; t >= 0 && found < levels; --t)
            if (bid_[t].count) { sum += bid_[t].total_qty; ++found; }
    else
        for (int32_t t = best_ask_; t <= max_tick_ && found < levels; ++t)
            if (ask_[t].count) { sum += ask_[t].total_qty; ++found; }
    return sum;
}
```

Each level already keeps a running `total_qty` (maintained by add/cancel/modify), so a level's depth is a single field read - we never walk the per-order lists here.

---

4. The timed path

Per market update we run exactly the loop a real strategy runs - read, decide, act - and time just that span:

```
auto t0 = Clock::now();
uint64_t bd = book.top_depth(Side::Buy, K);       // read
uint64_t ad = book.top_depth(Side::Sell, K);
uint64_t tot = bd + ad;
if (tot) {
    double obi = (double(bd) - double(ad)) / double(tot);   // decide
    if (obi > T)  book.submit(next_id++, Side::Buy,  book.best_ask(), 1, false, fills);  // act
    else if (obi < -T) book.submit(next_id++, Side::Sell, book.best_bid(), 1, false, fills);
}
auto t1 = Clock::now();     // <- signal-to-order latency
```

This is the number a trading desk cares about: **from seeing the book to the order leaving**.

---

5. What it measures

```
signal->order latency: p50 0 ns   p99 42 ns   p99.9 83 ns
fired 5866 orders (0.6% of 1,000,000 updates)
```

Two honest caveats:

- **p50 reads 0** because most updates *don't* fire (the imbalance sits inside ±0.30), and the read+compare alone is below this arm64 laptop's `steady_clock` resolution (~40 ns). It isn't literally zero - the clock just can't see under its own tick. On x86 Linux you'd use `rdtsc` (sub-nanosecond) and pin the thread to an isolated core to resolve it.
- **p99 / p99.9 (42 / 83 ns)** are the updates that *do* fire: two `top_depth` scans plus a `submit` that crosses and prints a fill. That's the real cost of acting on the signal.

The 0.6% fire rate is the threshold doing its job - trade only on conviction, not on noise.

---

6. Why this is the right capstone

It ties every earlier piece together into one pipeline: the **SPSC ring** feeds updates in, the **array book + id-map** apply them in ~40 ns, `top_depth` reads the state, and the **matching engine** executes the decision - all with zero hot-path allocation. That is the shape of a real low-latency trading loop, and the whole read-to-order path lands in tens of nanoseconds.

Not a trading strategy. OBI is illustrative - real use needs P&L backtesting, transaction costs, a decay/half-life study, and calibration on live data. What's demonstrated here is the **systems**

result: the signal-to-order path is fast and allocation-free. The alpha is a separate research problem; the plumbing is what this repo proves.